Architectural Migration, Binary Reconstruction, and Generational Analysis of the Heart Beat Engine in Dragon Quest IV and VII

The release of the PlayStation remake of *Dragon Quest IV: Chapters of the Chosen* (SLPM_869.16) represents a highly sophisticated exercise in legacy software migration and engine reuse. Developed by Heart Beat Inc., this title was constructed by refactoring and adapting the pre-existing 32-bit engine originally designed for *Dragon Quest VII: Eden no Senshitachi* (SLPM_865.00). The technical core of this adaptation lies in bridging a tile-based, 16-bit Super Famicom (SNES) architecture with a 32-bit polygonal rendering pipeline. To execute this, the engineers modified memory boundaries, constructed custom dynamic resource streams, and maintained an underlying coordinate system that preserved the 2D grid structure of the original game while projecting it onto a rotating 3D environment.

Architectural Divergence: The Data-Logic Gap

The transition of the game engine from a fixed-bank ROM cartridge architecture (typical of the Super Famicom era) to a disc-based dynamic system forced developers to establish strict control over the PlayStation's limited 2 MB of physical system RAM. This transition highlights a fundamental architectural divergence in memory management, registration width, and asset loading paradigms.

Super Famicom Cartridge Architecture

The Super Famicom (SNES) version relies on a fixed-bank memory architecture (such as HiROM or LoROM configurations). Under this model, the cartridge's physical ROM is mapped directly into the CPU's memory map, allowing instantaneous, low-latency access to code and data assets without the need for an intermediate load state. The logic of the Super Famicom engine is tightly coupled with the register-width limitations of the 16-bit Ricoh 5A22 CPU. Consequently, state tracking—such as character status, active event triggers, and combat AI parameters—is handled through extremely efficient, low-level bit manipulation.
For example, the game utilizes highly consolidated RAM buffers, such as the active party member status block located in RAM bank $7E. The primary data structures for active party members start at address $7e3925, with each character allocated a strict $3c (60-byte) block containing basic statistics, active status ailments, and equipment records. This compact layout was a direct consequence of physical memory limitations; early Famicom iterations were constrained to 2 KB of internal RAM, forcing subsequent 16-bit Super Famicom revisions to maintain highly optimized bit-packed matrices to track progression and save states.

PlayStation Disc-Based Streaming Architecture

The PlayStation hardware, operating on a 32-bit MIPS R3000 CPU, lacks direct ROM-mapping capabilities, relying instead on a physical CD-ROM drive using the ISO 9660 filesystem. Under this paradigm, assets cannot reside in fixed, instantly addressable hardware banks. The engine must implement a dynamic loading and streaming model, caching audio files, high-resolution sprite sheets, dialogue scripts, and 3D geometry blocks into system RAM on-the-fly.
The Heart Beat engine establishes a strict separation between the static "resident" engine code (which remains permanently buffered in memory) and the "volatile" expansion content that is streamed from the optical disc as needed. System memory is strictly partitioned to prevent overlaps, allocating dedicated segments to core executable routines, disk file systems, audio drivers, and dynamic heap buffers.

Feature or Paradigm	Super Famicom (SFC) Engine Architecture	PlayStation (PSX) Engine Architecture
CPU / Hardware Platform	16-bit Ricoh 5A22 (65C816 derivative)	32-bit MIPS R3000 CPU with GTE and MDEC
Primary Media / Interface	Solid-state cartridge (Direct Bus Mapping)	CD-ROM (ISO 9660 File System Streaming)
Memory Management	Bank switching (HiROM/LoROM)	Static partitioning with dynamic heap allocation
State Tracking Paradigm	High-efficiency 16-bit/8-bit register bit manipulation	32-bit structures, flag matrices, and dynamic VM variables
Asset Location & Access	Instantaneous access at fixed ROM offsets	Dynamic sector-based buffering via DMA Channel 4
Execution Loop Boundary	Interfaced via direct CPU interrupts and hardware timers	VSync callback registration and thread-driven scheduling

The Hidden Commonality and Game Loop Preservation

Despite the dramatic shift in underlying hardware and memory management paradigms, the core game loop remains the primary point of shared DNA between these generational versions. The logical framework developed for the 16-bit version was not discarded; rather, it was wrapped and preserved inside the 32-bit engine.

The AI Battle Kernel

The combat tactical engine represents the most direct logical bridge between the 16-bit and 32-bit implementations. In the original Famicom and Super Famicom versions, party automation was driven by a sophisticated decision-making kernel. The PlayStation remake wraps this identical 16-bit logic within a more complex, modern graphical interface and menu structure.
The underlying decision-making trees—which evaluate enemy targeting weights, spell selection priorities, healing thresholds, and status ailment vectors—use the same mathematical root logic. This shared logic is visible in how the game handles party configuration, equipment selection, and tactical overrides. During combat, the AI Battle Kernel evaluates a character's available choices against the active party tactics, such as "Fight Wisely," "Show No Mercy," or "Focus on Healing".
However, the PlayStation architecture solved a critical processing bottleneck. On 16-bit hardware, calculating the AI decision trees for an entire party introduced visible processing

latency. To mask this, the 16-bit engine began its AI calculations on the command menu itself; the calculations were performed as the menu faded out, hiding the processing delay from the player. The 32-bit R3000 CPU eliminated this bottleneck, allowing the engine to calculate AI decisions dynamically at the exact millisecond a character's turn begins. This enables real-time adaptation to mid-round status changes, such as an ally taking a critical hit.

Asset Mapping Continuity

Although the storage medium changed from raw cartridge banks to ISO files, the internal structure and progression of game data tables remain mirrored. The index layout of the game's core databases—specifically NPC loader templates, weapon and armor parameter indices, item behavior arrays, shop configuration blocks, and monster encounter tables—follows an identical logical progression.

A direct cross-reference of the database schemas reveals that the offset logic for the "bestiary index" and "script triggers" matches the progression of the original Super Famicom records. This is demonstrated by the database footprint of the modernized SNES port projects (such as dq4r-info), which maintain highly structured arrays for monsters, items, shops, and spells that align with the original data indexes :

Data Schema Type	Record Count	Structural Array Size (Bytes per Record)	Target System Integration
Monsters	50	8	HP, EXP, Gold, ATK, DEF, AGI parameter maps
Items	128	2	Price, equipment eligibility, and item type tables
Shops	180	2	Merchant inventory and price index configurations
Spells	50	6	MP cost, element type, and target-range parameters
Encounters	7,005	6	Map sector-based monster group indexing
Characters	16	2	Base statistics, growth tables, and class vectors

Scripted Events and 2D Grid Preservation

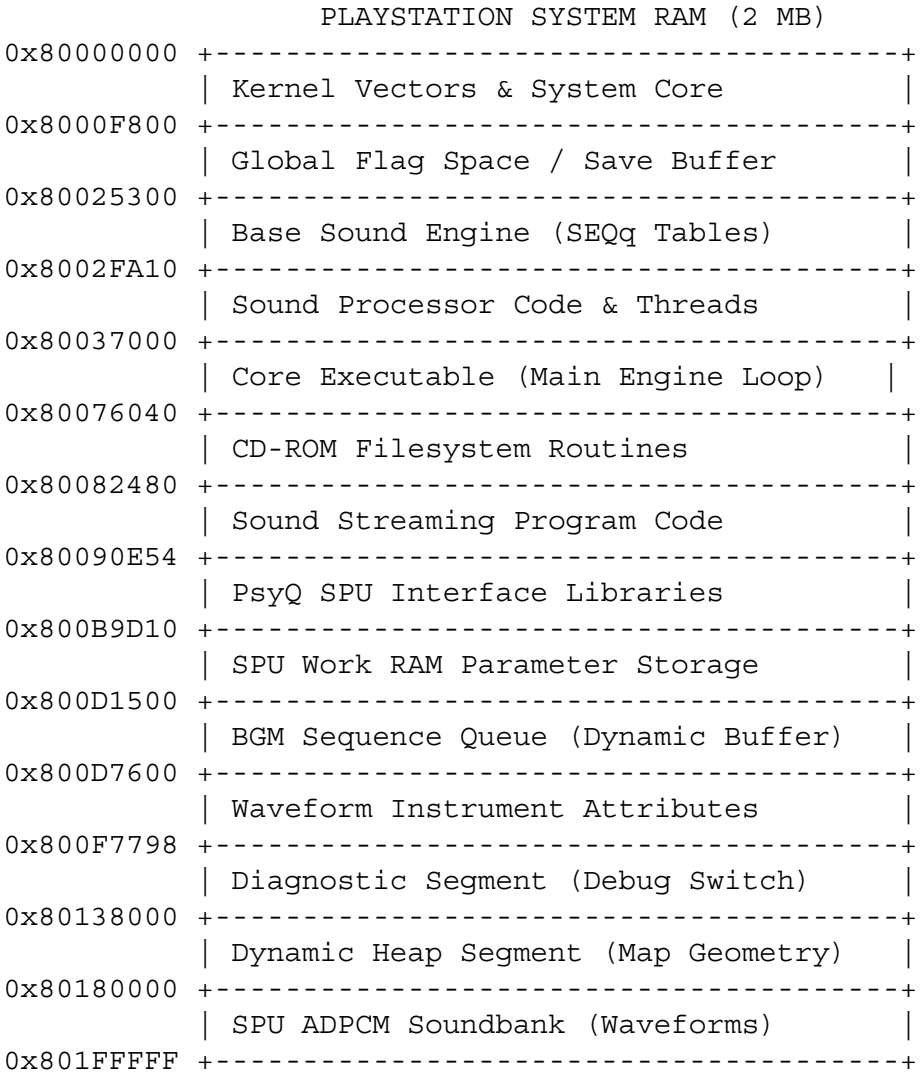
The movement of NPCs, the execution of cutscenes, and the progression of the game's chapter-based narrative rely on a "triggered event" system. Although the PlayStation engine renders environments using a 3D isometric camera that can rotate in 90-degree increments, the underlying event triggers are mapped to the same 2D coordinate grid that defined the original 16-bit map layouts.

Collision detection, event activation vectors, and NPC walking paths are evaluated on this flat 2D tile grid. The engine's renderer translates these 2D logical positions into 3D camera-space coordinates, proving that the underlying logic remains strictly decoupled from the modern 3D

presentation layer.

Memory Partitioning and Heap Allocation Mapping

To support this hybrid 2D/3D presentation on the PlayStation, Heart Beat bypassed generic operating system APIs to implement a custom, highly optimized memory map. The 2 MB (2048 KB) of system RAM is partitioned into static resident boundaries and dynamic heap pools. Static analysis of the primary executables shows that the base engine—including MIPS CPU instructions, rendering pipelines, dialogue interpreters, and disc system interfaces—is loaded into the lower bounds of memory, specifically starting at the compiler entry point 0x80037000. Immediately above this core executable segment lies the game state tracking matrix and the sound processing subsystem.



Dynamic File Streaming

Transitions between different regions, world maps, and towns require real-time file streaming to

avoid out-of-memory crashes. The core function managing these optical seek and read operations is located at address 0x80076040 (open_file). This routine serves as the primary driver interface for the PlayStation CD-ROM controller. When a player crosses a map transition boundary, open_file invokes a secondary, non-blocking background loader at address 0x80082598 (mopen_file).
The mopen_file routine accepts a single parameter passed through CPU register $a0, which points to a file descriptor structure containing the target Logical Block Number (LBN), sector length, and destination address in the dynamic heap. Rather than copying massive files in a single frame—which would stall the rendering loop and cause frame rate drops—mopen_file reads the data incrementally, sector-by-sector, in standard 2048-byte chunks.
For map instances, the sub-loader identifies the target asset resource type as 0x13, which triggers a multi-frame DMA streaming operation. The system uses DMA Channel 4 to transfer data directly from the CD-ROM controller to the dynamic map geometry heap starting at address 0x80138000, bypassing the CPU.
This streaming approach introduces a notable rendering artifact on modern emulators. When the player opens the in-game menu, the engine captures a snapshot of the current framebuffer from VRAM and copies it to a dedicated background buffer in System RAM. Because of the system's strict memory constraints, this copy is captured at the game's native rendering resolution (typically 256 \times 224 or 320 \times 240 pixels).
While this transfer is seamless on original hardware, modern emulators running at upscaled resolutions experience a visual mismatch: high-resolution menu text and UI elements are overlaid on top of a low-resolution, blocky background snapshot, creating a blurry background menu effect.

Archive Structures and Compression Protocols

The game’s non-executable assets are stored inside a single, massive resource archive: HBD1PS1D.Q41 in *Dragon Quest IV* and HBD1PS1D.Q71 in *Dragon Quest VII*. This archive divides evenly into 2048-byte physical sectors, aligning with the CD-ROM drive hardware.
The first sector (offsets 0x00 through 0x800) contains the volume identification header, with the ASCII string hdb1ps1d.q41 (or hdb1ps1d.q71) located at offset 0x400. The remaining sectors follow a strictly aligned block system.

Block Header Architecture

Every primary data block inside the archive begins with a 16-byte header that defines its structural boundaries :

- **Offset 0x00 (4 bytes):** Sub-block Count. Specifies the number of asset sub-blocks contained within the block.
- **Offset 0x04 (4 bytes):** Sector Count. The total number of physical 2048-byte sectors occupied by the block on the disc.
- **Offset 0x08 (4 bytes):** Total Data Length. The unpadded byte length of the raw, concatenated payload.
- **Offset 0x0C (4 bytes):** Reserved/zero padding.

Following this primary block header is a sequence of 16-byte sub-headers, located at offset 0x10 + (i \times 16) (where i represents the sub-block index) :

- **Offset 0x00 (4 bytes):** Compressed Data Size. The physical size of the sub-block

payload on the disc.

- **Offset 0x04 (4 bytes):** Uncompressed Data Size. The absolute RAM allocation required when decompressed.
- **Offset 0x08 (4 bytes):** System Flag.
- **Offset 0x0C (2 bytes):** Compression Flag. If this value is 1280 (0x0500), the payload is compressed using Heart Beat's custom LZSSO algorithm.
- **Offset 0x0E (2 bytes):** Resource Type Identifier.

LZSSO Decompression Protocol

When the engine encounters a compression flag value of 1280, it routes the payload to the software decompressor. The LZSSO decompressor initializes a sliding dictionary ring buffer of 4096 bytes, pre-filled with zero (0x00) bytes.
The compressed input stream is read as a sequence of control blocks, starting with a 1-byte control word C. The decompressor parses this control byte bit-by-bit from the least significant bit (b_0) to the most significant bit (b_7) to process the payload :

\text{Bit } b_i \text{ of control byte } C \implies \begin{cases} 1 & \text{Literal Byte Copy} \\ 0 & \text{Dictionary Reference (16-bit Descriptor } W) \end{cases}

Literal Byte Copy (b_i = 1)

The decompressor reads the next raw byte directly from the input stream, writes it to the destination output buffer in RAM, and appends it to the sliding dictionary history window :

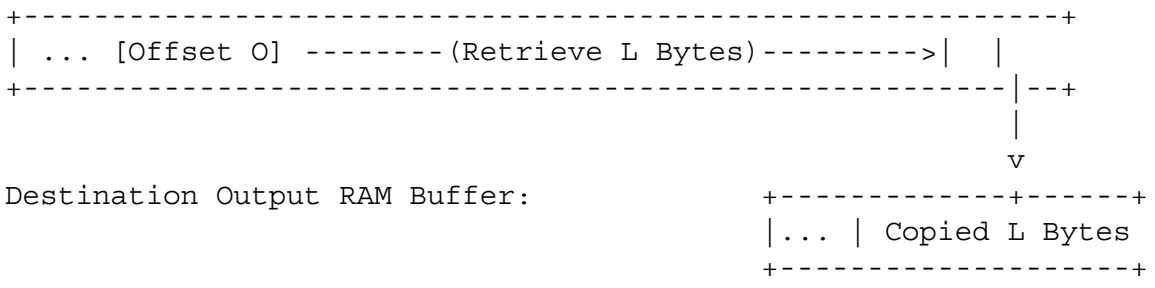
\text{Output} \leftarrow \text{Input}[I], \quad D \leftarrow D+1, \quad I \leftarrow I+1

Dictionary Reference (b_i = 0)

The decompressor reads a 2-byte (16-bit) descriptor word, W, from the input stream :
W \leftarrow (\text{Input}[I] \ll 8) \mid \text{Input}[I+1], \quad I \leftarrow I+2
This 16-bit word is unpacked to extract the history ring-buffer offset, O, and the repeating match length, L :
O \leftarrow W \gg 4 L \leftarrow (W \ \& \ \text{0x0F}) + 3
The decompressor then copies L bytes from the sliding history buffer starting at offset O directly into the active destination RAM buffer, updating the dictionary ring buffer with the copied bytes.

```
Compressed Input Stream:
+--------------+-----------------------+--------------------+
| Control Byte | 16-bit Descriptor (W) | Literal Byte (0xXX)|...
+--------------+-----------------------+--------------------+
  |
  +-- Bit b_i = 0 (Dictionary Reference Flag)
  |    - Offset (O) = W >> 4
  |    - Length (L) = (W & 0x0F) + 3
  |
  +-- Bit b_i = 1 (Literal Byte Copy Flag)
       - Read next raw byte directly to Output Buffer
       - Append raw byte to 4096-byte Sliding Ring Buffer

Sliding Ring Buffer (4096 Bytes, Initialized to 0x00):
```



The choice of the LZSSO algorithm represents a deliberate technical trade-off. Because the PlayStation's CPU lacks a dedicated hardware decompression coprocessor, real-time decompression of heavily compressed assets could introduce significant processing latency. LZSSO minimizes CPU overhead by relying on simple bitwise shifts and memory copies. By accepting slightly lower compression ratios compared to more complex algorithms, the engine is able to decompress graphics, scripts, and map data on-the-fly without causing frame rate stutter or disrupting the interrupt-driven sound processing threads.

Recommended Deep Dive Methodology for Engine Reconstruction

For developers working on engine reconstruction, ROM hacking, or cross-compatibility translation projects, the primary engineering effort should focus on three specific areas of binary mapping and analysis.

Binary String Mapping and Dialogue Encoding

The dialogue system of the Heart Beat engine evolved from the low-level, table-driven system used in the 16-bit Super Famicom releases. In *Dragon Quest III* and *Quest VI* for the Super Famicom, the game utilizes a custom Huffman compression scheme to unpack dialogue text on-the-fly. The character table (e.g., dq3 jp.tbl or dq6 jp.tbl) maps binary codes to Japanese glyphs, utilizing the byte sequence 00AE (or 00 depending on the context) as a strict string terminator.
The PlayStation version utilizes Huffman-coded Type 39 blocks, which are addressed via offset index tables inside Type 32 blocks. Comparing the character mapping table structures reveals how the core string assets were adapted. On the PlayStation, dialogue sequences use a more complex, control-character-driven markup system designed to handle multi-language formatting and dynamic name injection.

```
Super Famicom Dialogue Call (16-bit space optimization):
  JSR DoDialogLoBlock   ; Call dialogue routine (pushes Return PC to
Stack) [span_48](start_span)[span_48](end_span)
 .byte $0B              ; Inline Byte Index: Tells routine which string
to load [span_49](start_span)[span_49](end_span)
                        ; Routine reads byte at, increments
                        ; Return PC by 1, and returns execution past
the inline byte.

PlayStation Dialogue Command VM Instruction:
```

```
  C021A0 <offset> <dialogId>  ; Virtual Machine instruction to load
stream
```

When comparing character tables, developers should analyze the specific control codes and formatting markers used by the dialogue parsers across both architectures.

Functionality	Super Famicom (SFC) Character / Control Codes	PlayStation (PSX) Character / Control Codes
String Termination	00 or 00AE	0000
Line Break / Reset	Standard table-mapped line break controls	7f02 (Newline + Tab cursor reset)
Name Decorator Prefix	Hardcoded RAM index lookup triggers	7f04 followed by actor ID byte 7fXX
Blinking Cursor Pause	Pause wait timers	7f0a (Wait for input) / 7f0b (Reverse wait)
Dynamic Gold Variable	Low-level accumulator decimal conversions	7f15 (Dynamically inserts gold count)
Conditional VM Command	Hardcoded branching assembly scripts	C021A0 <FFF0> <key> (Evaluates event flags)
Text Translation Wrap	Custom DTE/MTE compression buffers	@aName@b... @c0@ / @c1@ markup envelopes

By mapping these control structures, developers can build translation tools (such as the dq4psxtrans Python toolkit) that dynamically replace Japanese strings with formatted English text, automatically managing line breaks and character name variables.

State Logic and AI Decision Weights

The mathematics used to determine combat AI decisions are highly consistent across both platforms. Comparing the MIPS disassembly from the PlayStation executable with the 65C816 assembly routines from the Super Famicom version reveals a shared logical lineage.
The mathematical weights used to prioritize spells and assess targets use the same logic. On the 16-bit Super Famicom CPU, these evaluations are executed using 8-bit or 16-bit register instructions:

```
; Super Famicom Assembly Example: Target HP Evaluation
LDA $7e3925+HP_OFFSET, X  ; Load active character's current HP
CMP #$0014                ; Compare HP against critical threshold (20
HP)
BCS SkipCriticalHealing   ; If HP is above threshold, branch to
standard actions
JSR CastCriticalHeal      ; Otherwise, prioritize immediate healing
spell
```

On the PlayStation, this logical tree is translated to the MIPS R3000 instruction set, running within the main execution segment of memory. While written in 32-bit assembly, the logic follows the same structure:

```
; PlayStation MIPS R3000 Disassembly Example: Target HP Evaluation
lw   v0, 0x10(a0)         ; Load active character's HP from status
block in heap
```

```
li   v1, 20               ; Load critical HP threshold value (20)
slt  v1, v0, v1           ; Test if current HP is less than 20
beqz v1, SkipCriticalHeal ; If false, branch to standard combat logic
nop                       ; Branch delay slot padding
jal  CastHealSpell        ; Otherwise, jump and link to critical
healing routine
```

By tracing these calculation pathways in a debugger (such as No$psx or Ghidra), developers can identify the specific logic weights that govern combat decisions. Adjusting these registers allows modders to modify AI behavior, correcting quirks in automated character decision-making or rebalancing combat mechanics.

Battle Background Handling: Decoupling View from Logic

The architectural differences in battle background rendering illustrate how the engine separates "view" from "logic."
On the 16-bit Super Famicom, rendering backgrounds is constrained by VRAM size and DMA bandwidth, often forcing the engine to use a simple static image or a plain black screen background to save resources. The code that handles these backgrounds is tightly integrated with the core combat loop; background values are loaded directly into the console's registers during screen transitions to set the hardware display mode.

```
Super Famicom Combat Initialization:
  [Core Combat Loop] ---> --->

PlayStation Double-Buffered Render Pipeline:
  [Core Combat Loop (Logic)] ---> --->

|

v
  <--- <---
```

The PlayStation engine, by contrast, treats battle backgrounds as distinct, page-aligned graphic assets. During combat transitions, the engine's rendering thread separates background drawing from game state calculations.
The background coordinates and textures are processed through the standard 3D pipeline, mapped using perspective projection via the GTE, and pushed to the GPU's Ordering Table (OT) as independent primitive draw commands. This design ensures that the underlying battle logic remains unaffected by visual rendering tasks, allowing developers to change background visuals without breaking combat code.

Narrative Persistence: Event Flag Matrices and Dynamic Streaming

To maintain a persistent narrative across multiple in-game chapters, generations, and physical world states without exhausting the system's memory limits, Heart Beat implemented a

compact, bit-packed event flag matrix.

Tracing Progression Flags in RAM

Live memory tracking shows the core game progress flags mapped within the static RAM range 0x8000F800 through 0x80010000. This segment acts as a dense bit-matrix where individual bits represent Boolean states for completed quests, recruited characters, and opened chests.
When a narrative event occurs, the engine updates this matrix using its state-writing routines. For example, debug and bypass parameters write directly to this block; the "Save Anywhere" codes modify bytes in the range 0x8000F83C through 0x8000F846, while inventory validation bypasses write to address 0x80069BCA to override standard equipment validation checks.
During execution, the engine evaluates these state changes using a conditional evaluation loop :

```
sb  v1, $0(a1)          ; Store updated state byte to flag pointer
...
beq v0, a0, $8001129C   ; Conditional branch evaluating flag criteria
```

To manipulate these state transitions during reverse engineering, this conditional branch instruction can be patched in memory with a No-Operation (nop) instruction to prevent the jump, or replaced with an unconditional jump (j $8001129C) to force the game state to evaluate as true.

Dynamic Town Streaming and Flag-Driven Model Selection

The engine uses this progress flag matrix to dynamically stream 3D models and construct environments in real-time. The most prominent example is the Immigrant Town, which dynamically alters its physical layout, loaded buildings, and active NPCs based on the active population flag.

```
Boundary Collision Trigger ---> Suspend World Map Renderer ---> Query
Progress Flag: 0x8000F800

|

v
mopen_file (0x80082598) Dynamic Stream <-- Retrieve Sector Addresses
<-- Offset Modifier Lookup
|
+--- 3D Mesh qQES   -> RAM Heap 0x80138000
+--- NPC Sheets TIM -> VRAM Page Cache (X >= 512)
```

When the player triggers a boundary transition into the town, the engine suspends the world map renderer and queries the progress flag range 0x8000F800. The returned state byte acts as an offset modifier for the asset lookup tables stored in the main executable, routing the sub-loader to the appropriate structural phase.
This flag-to-asset translation function retrieves the designated sector addresses from the physical archive and passes them directly to mopen_file (0x80082598). The assets are streamed into the dynamic heap at 0x80138000, ensuring that assets from other phases are never loaded. This architecture keeps memory usage well within the system's 2 MB physical

limit.

Synthesis and Architectural Conclusions

The comparison of the 16-bit and 32-bit versions of the Heart Beat engine illustrates how developers successfully adapted legacy software to more advanced hardware. By separating core gameplay logic from visual presentation systems, the engineers preserved the complex design of the original games while introducing modern 3D graphics.
For emulator developers and reverse engineers, this generational continuity offers a valuable framework. The consistency of the underlying database structures, AI logic weights, and grid-based coordinate mapping shows that 32-bit remakes can be analyzed using the same logical patterns as their 16-bit predecessors. Understanding how these systems are linked simplifies the process of developing modification tools, translation patches, and platform ports, supporting the preservation of these classic RPGs.

Works cited

1. TheAnsarya/dq4r-info: Dragon Quest IV Remix - SNES port of Dragon Warrior IV NES using the DQ3r engine - GitHub, https://github.com/TheAnsarya/dq4r-info 2. Dragon Quest III (SNES)/RAM map/Party Members - Data Crystal - TCRF.net, https://datacrystal.tcrf.net/wiki/Dragon_Quest_III_(SNES)/RAM_map/Party_Members 3. Limited Inventory of Early DQ is Good Game Design : r/dragonquest - Reddit, https://www.reddit.com/r/dragonquest/comments/1nt0dzs/limited_inventory_of_early_dq_is_good_game_design/ 4. Artificial Intelligence - Dragon Quest Wiki, https://dragon-quest.org/wiki/Artificial_Intelligence 5. Defeating all bosses in the FC version of Dragon Quest IV using only AI | frenchbread - note, https://note.com/frenchbread/n/na0028587e6d3?hl=en 6. First time players, 10 hours in, how are you using the tactics? : r/dragonquest - Reddit, https://www.reddit.com/r/dragonquest/comments/1qbzac8/first_time_players_10_hours_in_how_are_you_using/ 7. Dragon Quest VII: Sound Engine Analysis - GitHub Gist, https://gist.github.com/loveemu/582ff7683a797b7048b6 8. Dragon „Hackst“ IV | Markus Projects, http://markus-projects.net/dragon-hackst-iv/ 9. ButThouMust/dq6-sfc: Font and script dumpers for Dragon Quest VI and III for Super Famicom. - GitHub, https://github.com/ButThouMust/dq6-sfc 10. Dragon Quest III (SNES)/ROM map - Data Crystal - TCRF.net, https://datacrystal.tcrf.net/wiki/Dragon_Quest_III_(SNES)/ROM_map 11. Attempt at Python tooling for the Dragon Quest 4 PSX Translation - GitHub, https://github.com/mwilkens/dq4psxtrans 12. mwilkens (Mandy Wilkens) · GitHub, https://github.com/mwilkens